Snowpark Connect - Getting Started

Author: *Jerry Tang*

Last significant update: *Mar 7, 2025*

[Snowpark Connect - Getting Started](#)
- [Background](#)
  - [Key Benefits](#)
  - [Limitations](#)
- [Prerequisites](#)
- [Running Spark code on Snowflake](#)
  - [Installation (all languages)](#)
  - [snowpark-submit (python file, jar)](#)
  - [PySpark (Jupyter Notebook)](#)
  - [spark-shell (Scala)](#)
  - [(Optional) Cleanup and uninstall](#)
- [Appendix](#)
  - [Supported Data Sources and Formats](#)
  - [Pyspark Examples](#)
    - [Verified](#)
    - [Work in progress](#)
    - [Obsolete](#)
    - [Out of scope](#)
  - [Snowflake Notebook (blocked on Python 3.10 support)](#)

Background

Historically, migrations from Spark to Snowpark have not been easy; users have to manually convert, validate and migrate their code to Snowpark. With the **Snowpark Connect entering Private Preview**, we'll allow users to just **re-target their existing Spark workloads** starting with Dataframe API on Snowflake and **run in Snowflake compute as-is.** This should dramatically reduce the time, cost, and operational complexity of migrating Spark workloads.

Key Benefits

- **Faster Migrations**: no need to rewrite spark and data validation just works across spark and snowpark

- **Lower Learning Curve**: Teams can stay in their familiar Spark environment, easing adoption.
- **No Vendor Lock-In**: Customers can “plug and play” between Spark engines, minimizing friction.
- **Multiple languages supported:** We support PySpark, Java, Scala and Spark SQL

Limitations

- Our initial scope focuses on data frame API(s) and requires 3.4+ Spark clients. The solution will work in PrPr if your data is in Snowflake or you are willing to move it to Snowflake stages.
- We will not support any RDD-based workload as part of the initial scope

Prerequisites

Set up your *[spark-connect]* connection at *.snowflake/connections.toml*.

Install Snow CLI if you haven’t already, instructions [here]. Use `snow connection add` to set up a connection for `spark-connect`.

Your connections.toml file should have a section that looks like this, feel free to point to your own account:

```
Unset
[spark-connect]
host="sfctest0.snowflakecomputing.com"
port=443
account="sfctest0"
user="abc"
password="&&&&&&"
protocol="https"
warehouse="testwh_sas"
database="testdb_sas"
schema="public"
```

Make sure you have Java of the same architecture as Python. I.e.
If Python is based on ARM architecture then you need Java based on ARM architecture.
If Python is based on x86_64 architecture then you need Java based on x86_64 architecture.

Set the JAVA_HOME environment variable for the java install you have.

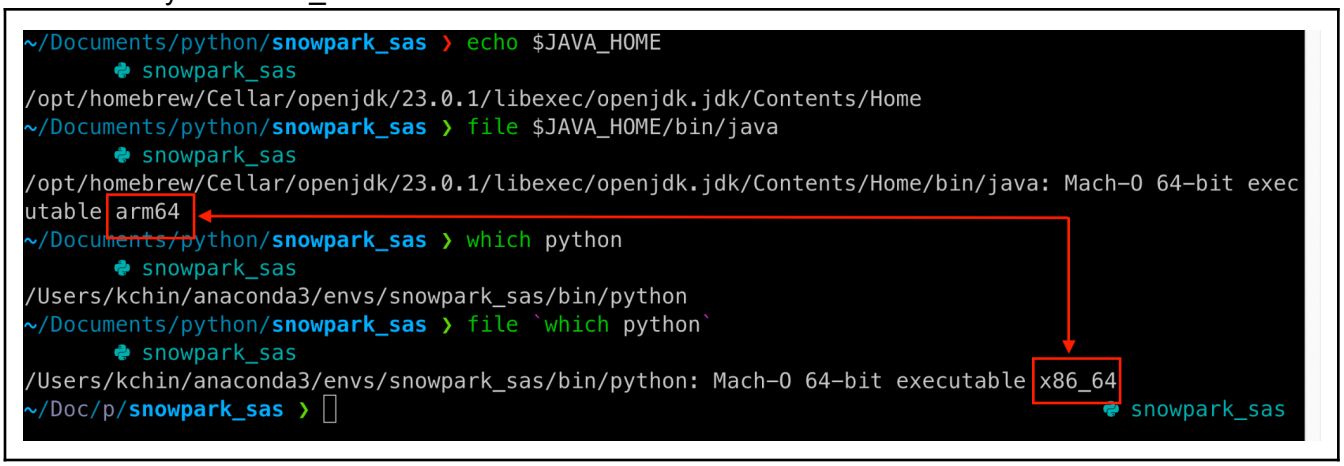
How to check architecture of Python and Java on Mac:

```
Unset
Python:
(.venv) igarish@XY5990KYVC sas % which python
/Users/igarish/git/sas/.venv/bin/python
(.venv) igarish@XY5990KYVC sas % file /Users/igarish/git/sas/.venv/bin/python
/Users/igarish/git/sas/.venv/bin/python: Mach-O 64-bit executable arm64

Java:
(.venv) igarish@XY5990KYVC sas % echo $JAVA_HOME
/Users/igarish/Library/jdk-11.0.26+4/Contents/Home
(.venv) igarish@XY5990KYVC sas % file $JAVA_HOME/bin/java
/Users/igarish/Library/jdk-11.0.26+4/Contents/Home/bin/java: Mach-O 64-bit
executable arm64
```

Note - Architectures ***must be matching*** between Python and JAVA.
Example of Mismatched Binaries (misconfiguration)

- Java - arm64
- Python - x86_64



Debugging Tips/Verification Sequence:

- Double Check your ENV variable JAVA_HOME setting
  - `$JAVA_HOME/bin/java --version`
- Java and Python must be the same architecture (see above)
  - `file $JAVA_HOME/bin/java # x86_64`
  - `file `which python` # x86_64`
- Use the most recent Snowpark Connect package WHL file from here

- Confirm `pyspark` is correctly working for ***local*** execution– > e.g.:
  - `python pyspark-array_string.py` (PY here)
  - Does not use any snowflake connectivity (local)
  - PY file is unchanged (from github repo)
- Test TOML connection is correct for ***remote*** execution – > e.g.:
  - `spark-submit pyspark-array_string.py` (PY here)
  - Uses `connection.toml` to authenticate to Snowflake
  - PY file is unchanged (from github repo)
- Test connection sequence below in a IPYNB notebook
  - Confirms the notebook works interactively
  - Using jupyter or VS code

Running Spark code on Snowflake

You can run spark code on snowflake using one of the following methods:

1. Command line: snowpark-submit
2. Jupyter Notebook (PySpark)
3. spark-shell (scala)
4. Snowflake Notebook (coming soon)

Installation (all languages)

1. (Optional) Create a Python environment to test drive SAS, we use Conda in this guide.

```
Unset
conda create -n xxxx pip python=3.11
conda activate xxxx
```

2. Install the Snowpark Connect package (download from the drive that was shared with you or check your email)

```
Unset
pip install -U --force-reinstall snowpark_connect-0.2.0-py3-none-any.whl
```

snowpark-submit (python file, jar)

`spark-submit` is supported through a script shipped with the python package called `snowpark-submit`

1. Run your PySpark script using the runner script that comes with Snowpark Connect package (plenty of examples from here)

```
Unset
snowpark-submit your_spark_submit_script
```

Jupyter Notebook (PySpark)

1. Follow installation instructions in Installation (all languages)
2. Enable Spark Connect Mode by adding the following preamble to the beginning of your Notebook or Python script.

```
Python
import os
from snowflake import snowpark_connect
from pyspark.sql import SparkSession

os.environ["SPARK_CONNECT_MODE_ENABLED"] = "1"
remote_url = "sc://localhost:15002"
os.environ["SPARK_REMOTE"] = remote_url
snowpark_connect.start_session(remote_url=remote_url)  # Start the local
Snowflake Spark Connect session

spark = SparkSession.builder.appName("SnowparkConnect").getOrCreate()
```

3. Write pyspark code in Jupyter Note as you normally would. You can try the extremely simple one as below or use Pyspark examples or start exploring various guides from sites such as Kaggle.

```
Python
from pyspark.sql import Row

df = spark.createDataFrame([
                Row(a=1, b=2.),
                Row(a=2, b=3.),
                Row(a=4, b=5.),])
print(df.count())
```

spark-shell (Scala)

1. Follow installation instructions in Getting started (Installation all languages)
2. Follow the instructions in the Spark documentation to install `spark-connect-repl`
3. Start a `snowpark-connect` session in your bash environment

```
Python
$ snowpark-connect start
```

    a. `snowpark-connect stop` ends the session
    b. `snowpark-connect restart` restarts the session (in case of timeout)
    c. You may also check if the session is alive with `snowpark-connect check`

4. Start the spark-shell via the `spark-connect-repl`

```
Unset
$ SPARK_REMOTE="sc://localhost:15002" spark-connect-repl
```

5. Write your scala code, for example.

```
Java
import scala.math.random

import org.apache.spark.sql.SparkSession

/** Computes an approximation to pi */
object SparkPi {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession
      .builder()
      .appName("Spark Pi")
      .getOrCreate()
    val slices = if (args.length > 0) args(0).toInt else 2
    val n = math.min(100000L * slices, Int.MaxValue).toInt // avoid overflow
    val count = spark.sparkContext.parallelize(1 until n, slices).map { i =>
      val x = random() * 2 - 1
      val y = random() * 2 - 1
      if (x*x + y*y <= 1) 1 else 0
    }.reduce(_ + _)
    println(s"Pi is roughly ${4.0 * count / (n - 1)}")
    spark.stop()
  }
}
```

(Optional) Cleanup and uninstall

1. Remove the Conda environment

```
Unset
conda env remove -n xxxx
```

Appendix

Supported Data Sources and Formats

- Data Sources
    - Local file (Yes)
    - Tables on Snowflake (Yes)
    - Cloud Storage
        - Amazon S3 (Yes)
        - Azure Blob Storage (Yes)
        - Google Cloud Storage (Yes)
    - RDBMS (No, ETA March 2025)
    - Messages Queues (No)
    - NoSQL Database
        - Cassandra (No)
        - MongoDB (No)
    - HDFS/Hive (No)

- Formats
    - Text (Yes)
    - CSV (Yes)
    - JSON (Yes)
    - Parquest (Yes)
    - ORC (No)
    - Avro (No)
    - XML (No)

- Catalog and its APIs
  Support only Snowflake as the catalog provider, and the following catalog APIs:
    - pyspark.sql.connect.catalog.Catalog.cacheTable
    - pyspark.sql.connect.catalog.Catalog.clearCache
    - pyspark.sql.connect.catalog.Catalog.currentCatalog
    - pyspark.sql.connect.catalog.Catalog.currentDatabase
    - pyspark.sql.connect.catalog.Catalog.databaseExists
    - pyspark.sql.connect.catalog.Catalog.dropGlobalTempView
    - pyspark.sql.connect.catalog.Catalog.dropTempView
    - pyspark.sql.connect.catalog.Catalog.getDatabase
    - pyspark.sql.connect.catalog.Catalog.getTable
    - pyspark.sql.connect.catalog.Catalog.isCached
    - pyspark.sql.connect.catalog.Catalog.listCatalogs
    - pyspark.sql.connect.catalog.Catalog.listColumns
    - pyspark.sql.connect.catalog.Catalog.listDatabases
    - pyspark.sql.connect.catalog.Catalog.listTables
    - pyspark.sql.connect.catalog.Catalog.recoverPartitions
    - pyspark.sql.connect.catalog.Catalog.refreshByPath
    - pyspark.sql.connect.catalog.Catalog.refreshTable
    - pyspark.sql.connect.catalog.Catalog.setCurrentCatalog
    - pyspark.sql.connect.catalog.Catalog.setCurrentDatabase
    - pyspark.sql.connect.catalog.Catalog.tableExists
    - pyspark.sql.connect.catalog.Catalog.uncacheTable
    - pyspark.sql.connect.catalog.Catalog.registerFunction ***(Not Implemented)***
    - pyspark.sql.connect.catalog.Catalog.listFunctions ***(Not Implemented)***
    - pyspark.sql.connect.catalog.Catalog.getFunction ***(Not Implemented)***
    - pyspark.sql.connect.catalog.Catalog.functionExists ***(Not Implemented)***
    - pyspark.sql.connect.catalog.Catalog.createExternalTable ***(Not Implemented)***
    - pyspark.sql.connect.catalog.Catalog.createTable (***Partial support except external table***)

- Iceberg Table
    - Snowflake Managed tables (Yes for Read/Write, No for Create/SaveAs)
    - Externally managed Iceberg Table (No)

Pyspark Examples

The open source pyspark examples contain 97 PySpark examples. They can be put into 4 categories:

- Verified (42)
- Work-in-progress (9)
- Obsolete (10)
- Out of scope (36)

Verified

1. pyspark-add-new-column.py
2. pyspark-add-month.py
3. pyspark-aggregate.py
4. pyspark-arraytype.py
5. pyspark-cast-column.py
6. pyspark-collect.py
7. pyspark-convert_columns-to-map.py
8. pyspark-count-distinct.py
9. pyspark-current-date-timestamp.py
10. pyspark-dataframe-flatMap.py
11. pyspark-dataframe.py
12. pyspark-date-string.py
13. pyspark-date-timestamp-functions.py
14. pyspark-datediff.py
15. pyspark-distinct.py
16. pyspark-drop-column.py
17. pyspark-explode-nested-array.py
18. pyspark-expr.py
19. pyspark-filter.py
20. pyspark-groupby-sort.py
21. pyspark-groupby.py
22. pyspark-join-two-dataframes.py
23. pyspark-left-anti-join.py
24. pyspark-orderby-groupby.py
25. pyspark-python-dataframe.py
26. pyspark-rename-column.py
27. pyspark-select-columns.py
28. pyspark-show-top-n-rows.py
29. pyspark-split-function.py
30. pyspark-sql-case-when.py
31. pyspark-string-date.py
32. pyspark-string-timestamp.py
33. pyspark-string-to-array.py
34. pyspark-struct-to-map.py
35. pyspark-time-diff.py
36. pyspark-timestamp-date.py
37. pyspark-types.py
38. pyspark-union.py
39. pyspark-unix-time.py
40. pyspark-update-column.py
41. pyspark-window-functions.py
42. pyspark-withcolumn.py

Work in progress

1. pyspark-column-operations.py
2. pyspark-explode-array-map.py
3. pyspark-filter2.py
4. pyspark-join.py
5. pyspark-lit.py
6. pyspark-pivot.py
7. pyspark-read-json.py
8. pyspark-udf.py

Obsolete

1. pyspark-array-string.py
2. pyspark-change-string-double.py
3. pyspark-drop-null.py
4. pyspark-filter-null.py
5. pyspark-fulter-null.py
6. pyspark-orderby.py failed
7. pyspark-read-csv.py
8. pyspark-repace-null.py
9. pyspark-shape-dataframe.py
10. pyspark-structtype.py
11. pyspark-when-otherwise.py

Out of scope

1. convert-column-python-list.py
2. currentdate.py
3. pandas-pyspark-dataframe.py
4. pyspark-broadcast-dataframe.py
5. pyspark-column-functions.py
6. pyspark-convert-map-to-columns.py
7. pyspark-create-dataframe-dictionary.py
8. pyspark-create-dataframe.py
9. pyspark-create-list.py
10. pyspark-dataframe-repartition.py
11. pyspark-empty-data-frame.py
12. pyspark-loop.py
13. pyspark-mappartitions.py
14. pyspark-maptype-dataframe-column.py
15. pyspark-parallelize.py
16. pyspark-partitionby.py
17. pyspark-print-contents.py
18. pyspark-range-partition.py
19. pyspark-rdd-actions.py

20. pyspark-rdd-broadcast.py
21. pyspark-rdd-flatMap.py
22. pyspark-rdd-map.py
23. pyspark-rdd-reduceByKey.py
24. pyspark-rdd-to-dataframe.py
25. pyspark-rdd-wordcount-2.py
26. pyspark-rdd-wordcount.py
27. pyspark-rdd.py
28. pyspark-repartition-2.py
29. pyspark-repartition.py
30. pyspark-row.py
31. pyspark-sampling.py
32. pyspark-sparksession.py
33. python-pandas.py
34. schema.py
35. spark-repartition-2.py
36. timediff.py

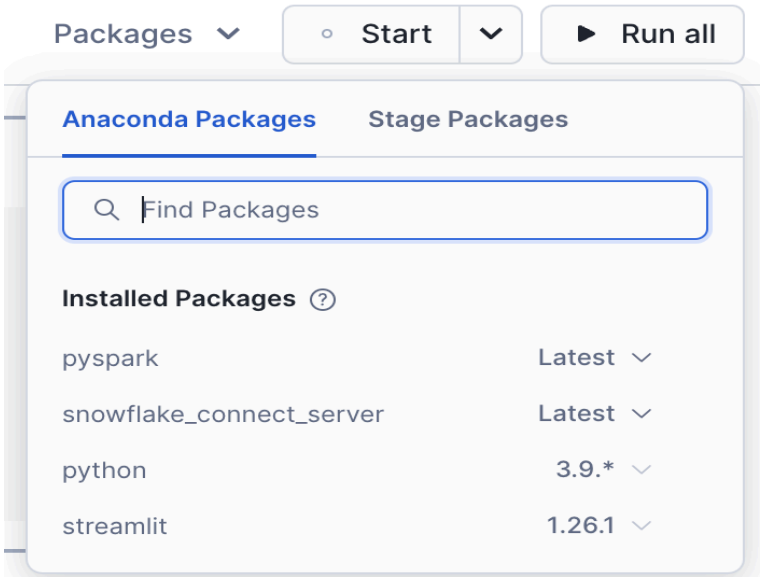
Snowflake Notebook (blocked on Python 3.10 support)

Prerequisites in lieu of setting up *.snowflake/config.toml*:

- DEV_VPN (until GS 9.3 goes live, ETA 02/18)
- Preprod/Prod Setup: we've verified Preprod8.
- Set the following account parameters.

```
Unset
-- Verified using account notebook_testaccount on preprod8
alter account notebook_testaccount set NOTEBOOK_ENGINE_VERSION =
'notebook_v1.55.2_py_3_11' parameter_comment = 'sas test on notebook';
alter account notebook_testaccount set ENABLE_GVISOR_NETWORK_NAMESPACE =
True parameter_comment = 'sas test on notebook';
alter account notebook_testaccount set udf_jail = 'gvisor'
parameter_comment = 'sas test on notebook';
```

1. From the package selector, install the following packages:
    a. *pyspark 3.5.3*
    b. streamlit 1.39.1
    c. python 3.10 or above
    d. *snowpark_connect*



2. Add the following preamble to the beginning of your Notebook.

```
Python
from pyspark.sql.connect.session import SparkSession as SparkConnectSession
from snowflake.snowpark.context import get_active_session
from snowflake.snowpark_connect.server import UnixDomainSocketChannelBuilder
from snowflake import snowpark_connect

snowpark_connect.start_session(unix_domain_socket="/tmp/sas-grpc.sock",
snowpark_session=get_active_session())
```

3. Do your Spark coding in the Notebook. You can try the extremely simple one as below or start exploring various guides from sites such as Kaggle.

```
Python
from pyspark.sql import Row

df = spark.createDataFrame([
                Row(a=1, b=2.),
                Row(a=2, b=3.),
                Row(a=4, b=5.),])
print(df.count())
```